

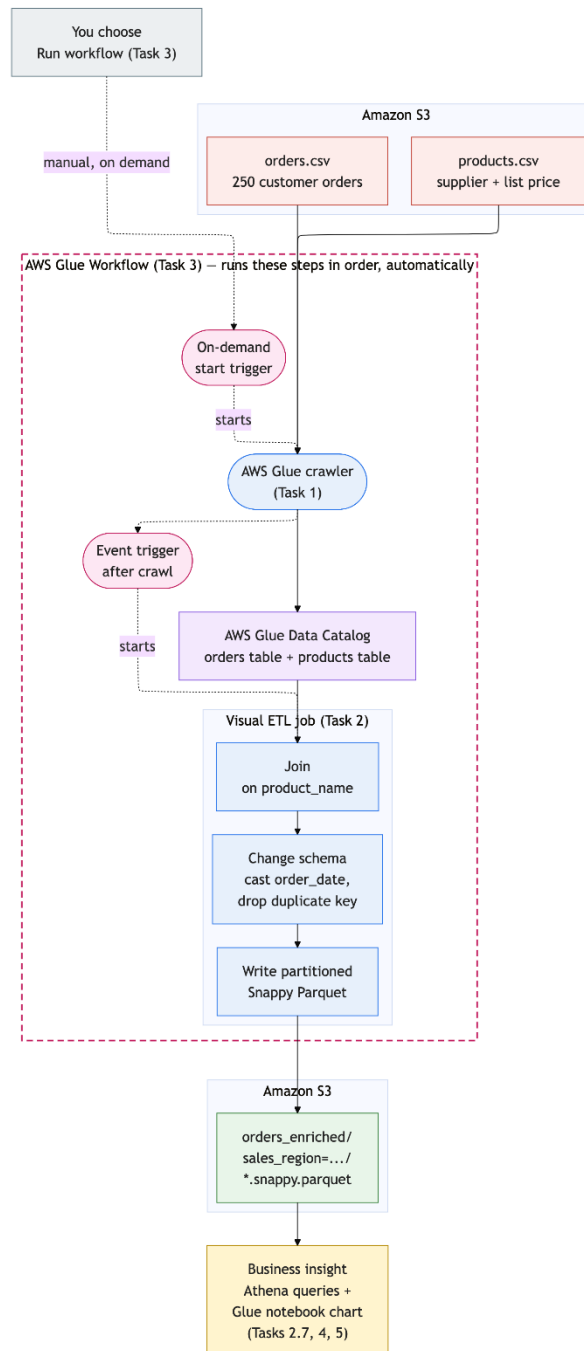
Lab overview

You work as a data engineer at **AnyCompany Retail**. Every day the business records new customer orders in one file, while the catalog team maintains supplier and list-price details for each product in a separate reference file. Whenever leadership asks *“which suppliers drive the most revenue?”* or *“how much margin are we giving away to discounts?”*, an analyst named **Mateo** combines the two files by hand in a spreadsheet. That manual matching is slow, easy to get wrong, and out of date the moment new orders come in - and Mateo would rather spend his time on analysis than on copy-paste. Your job is to replace his spreadsheet with an automated, repeatable AWS Glue pipeline.

In this lab, you build a data pipeline with AWS Glue. Instead of running a single job by hand, you integrate the two datasets, orchestrate the whole pipeline so it runs as one unit, and process new data incrementally so each run handles only what changed.

You crawl AnyCompany’s two raw datasets in Amazon S3 (retail orders and a product reference table), **join** them in an AWS Glue Visual ETL job to produce an enriched table, and write the result as partitioned Apache Parquet. You then wrap the crawler and the ETL job in an AWS Glue **Workflow** driven by **triggers** so the pipeline runs end to end with one action. Next you enable **job bookmarks**, land a new batch of orders, and re-run the workflow to confirm only the new records are processed. Finally, you explore the result in an AWS Glue **interactive notebook** and chart it.

The pipeline you build moves data - from AnyCompany’s raw CSV files, through the catalog and the join, to a query-ready enriched table in the processed bucket:



In Task 3 you wrap the crawler and the ETL job in a **Workflow** so these steps run automatically, in order, as a single unit.

Objectives

By the end of this lab, you should be able to do the following:

- **Integrate** multiple datasets by joining two cataloged tables in an AWS Glue Visual ETL job.
- **Orchestrate** a crawler and an ETL job as a single AWS Glue Workflow using triggers.
- Process data **incrementally** with AWS Glue job bookmarks.
- **Visualize** processed data in an AWS Glue interactive notebook.

Technical knowledge prerequisites

- Basic SQL knowledge will be helpful but is not required.
 - No PySpark authoring is required; the Visual ETL canvas generates the code and the notebook cells are provided.
-

Lab environment

The lab resources need to create:

- An **Amazon S3 bucket** named **glue-raw-<account-id>-<region>** with three folders namely- orders, products, notebook and incoming.
- Another **Amazon S3 bucket** named **glue-processed-<account-id>-<region>** for your transformed output.
- An **AWS Glue Data Catalog database** named **<glue_db>**.
- An **AWS Glue service role** named **< glue-lab-service-role >** you assign to the crawler, ETL job, workflow, and notebook.

You build the crawler, ETL job, workflow, and notebook yourself.

glue-lab-service-role:-

Permissions to be added;

1. [AWSGlueServiceRole](#) --- AWS Managed
2. [glue-session-passrole](#)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Condition": {
        "StringEquals": {
          "iam:PassedToService": "glue.amazonaws.com"
        }
      },
      "Action": "iam:PassRole",
      "Resource": "arn:aws:iam::<AWS-ACCOUNT-NUMBER>:role/glue-lab-service-role",
      "Effect": "Allow"
    }
  ]
}
```

```
]
}
```

3. [lab-bucket-access](#)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": [
        "s3:GetObject",
        "s3:ListBucket",
        "s3:GetBucketLocation"
      ],
      "Resource": [
        "arn:aws:s3:::glue-raw-<AWS-ACCOUNT-NUMBER>-<REGION>",
        "arn:aws:s3:::glue-raw-<AWS-ACCOUNT-NUMBER>-<REGION>/*"
      ],
      "Effect": "Allow"
    },
    {
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket",
        "s3:GetBucketLocation"
      ],
      "Resource": [
        "arn:aws:s3:::glue-processed-<AWS-ACCOUNT-NUMBER>-<REGION>",
        "arn:aws:s3:::glue-processed-<AWS-ACCOUNT-NUMBER>-<REGION>/*"
      ],
      "Effect": "Allow"
    }
  ]
}
```

```
}  
]  
}
```

Task 1: Catalog two data sources

In this task, you create a single crawler that discovers **both** raw datasets and registers them as two tables in the Data Catalog. Cataloging both sources up front lets you join AnyCompany's orders to its product catalog in the next task.

Task 1.1: Create a crawler with two data sources

Create a single crawler that points at both of AnyCompany's raw datasets in Amazon S3.

3. In the AWS Management Console search box, enter

Glue

and then choose **AWS Glue**.

4. Choose the hamburger icon to open the left-side navigation, expand **Data Catalog** and choose **Crawlers**.
5. Choose **Create crawler**.
6. For **Name**, enter

raw-data-crawler

, and choose **Next**.

7. On the **Choose data sources and classifiers** page, leave **Not yet** selected for *Is your data already mapped to Glue tables?*, and then choose **Add a data source**.
8. In the **Add data source** dialog, configure the following:
 - **Data source**: keep **S3** selected.
 - **S3 path**: choose **Browse S3**, select the **glue-raw-<account-id>-<region>** bucket link, select the radio button next to the **orders/** folder, and then choose **Choose**.
 - Choose **Add an S3 data source**

The Data Sources list now includes one data source.

9. Choose **Add a data source** again to add the second source. Configure the following:
 - **Data source**: keep **S3** selected
 - **S3 path**: choose **Browse S3**, select the same **glue-raw...** bucket link, select the radio button next to the **products/** folder, and then choose **Choose**.
 - Choose **Add an S3 data source**

The Data Sources list now includes two data sources.

10. **Note:** Verify that you added **orders/** and **products/** as two separate data sources.

10. Confirm both S3 paths are listed, and then choose **Next**.

Task 1.2: Assign the service role and output database

Assign the crawler the IAM role to use and the Data Catalog database to write to.

11. On the **Configure security settings** page, for **Existing IAM role**, choose **glue-lab-service-role**, and then choose **Next**.

12. On the **Set output and scheduling** page, for **Target database**, choose **glue_db**.

13. Leave **Frequency** set to **On demand**, and then choose **Next**.

14. Review the settings, and then choose **Create crawler**.

Task 1.3: Run the crawler and inspect the tables

Run the crawler and confirm it registered both datasets as catalog tables.

15. On the crawler details page, choose **Run crawler**.

Wait for the **Status** in the Crawler runs panel to change from **Running** to **Succeeded**. It usually takes one to two minutes. Optionally, use the refresh icon in the Crawler runs area to notice more quickly when it is complete.

16. In the navigation pane, under **Data Catalog**, choose **Tables**.

Analysis: The single crawler created **two** tables - **orders** and **products** - because you gave it two data sources. A crawler classifies each distinct S3 prefix as its own table.

17. Choose the **orders** table link and review its schema.

It has nine columns. Notice that **order_date** was detected as **string**.

18. Choose **Tables** again from the left-side navigation, then choose the **products** table.

It has 3 columns: **product_name**, **supplier**, **list_price**.

Task complete: You cataloged both source S3 datasets as **orders** and **products** tables in Glue.

Task 2: Build an integration ETL job

Now you build a Visual ETL job that **joins** the two tables on **product_name**, enriching each order with its supplier and catalog list price, corrects the **order_date** type, and writes partitioned Parquet to the processed bucket. This is the enriched view AnyCompany's analysts have been assembling by hand - now produced automatically and the same way every time.

Task 2.1: Create the job and add two sources

Start a new Visual ETL job and add the two cataloged tables as its sources.

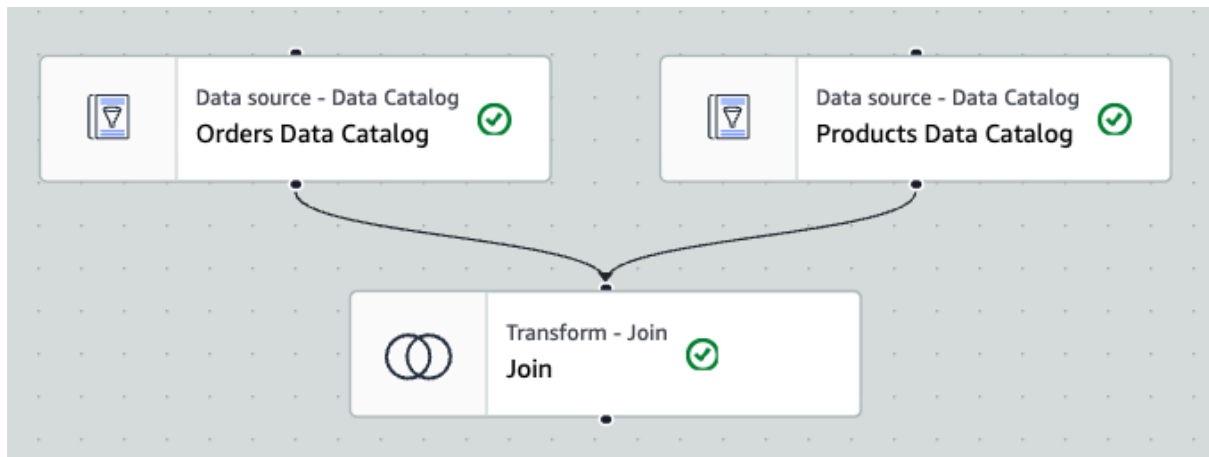
19. In the navigation pane, expand **Data Integration and ETL**, and then choose **Visual ETL**.

20. Choose **Visual ETL** to open a blank canvas.
21. In the top left of the screen, choose *Untitled job* next to the pencil icon and give it the name **orders-etl-job**.
22. In the **Job details** tab, for **IAM Role**, choose the existing **glue-lab-service-role...** IAM role.
23. In the **Visual** tab, choose **Add nodes** (the + icon in the top left of the canvas) to open the Add nodes menu if it is not already open.
24. On the **Sources** tab, choose **AWS Glue Data Catalog**. Select the node that now appears on the canvas and, on the **Data source properties** tab, set:
 - **Name:** **Orders Data Catalog**
 - **Database:** select **glue_db**
 - **Table:** select **orders**
25. Choose **Add nodes** (the + icon) again, then from the **Sources** tab add a second **AWS Glue Data Catalog** source, and set:
 - **Name:** Products Data Catalog
 - **Database:** select **glue_db**
 - **Table:** select **products**

Task 2.2: Join the two sources

Combine orders with product details by joining on the shared **product_name** column.

26. Choose **Add nodes (+)**. On the **Transforms** tab, choose **Join**.
27. Select the **Join** node.
 - For **Node parents**, select *both* **Product Data Catalog** and **Orders Data Catalog**.
 - For **Join type**, keep **Inner join** selected.
 - Under **Join conditions**, choose **Add condition** and set the condition to match the key in both tables:
 - Left (Products Data Catalog): **product_name**
 - Right (Orders Data Catalog): **product_name**
28. Choose **Save** to save the job.



Task 2.3: Clean up the schema

Cast **order_date** to a date type and drop the duplicate join column.

29. Choose **Add nodes (+)** and on the **Transforms** tab, choose **Change Schema**.
30. Select the transform node that was added to the canvas.
31. For **Node parents** verify the **Join** node is selected (select if it was not already added).
32. In the Change Schema (Apply mapping) area:
 - Locate the **order_date** source key and change the **Data type** from **string** to **date**.
 - Where the join produced a duplicate product name column (**.product_name**), select its **Drop** check box.
 - Confirm **supplier** and **list_price** from the products table are present.

These are the enrichment columns the join added.

33. Choose **Save**.

Task 2.4: Add the partitioned S3 target

Write the job's output to the processed bucket as Parquet, partitioned by **sales_region**.

34. Choose **Add nodes (+)**. On the **Targets** tab, choose **Amazon S3**.
35. Select the **Data target - S3 bucket** node that was added to the canvas and configure:
 - **Node parents**: ensure that the **Change Schema** (ApplyMapping - Transform) is selected.
 - **Format**: confirm **Parquet** is selected
 - **Compression Type**: confirm **Snappy** is selected
 - **S3 Target Location**:
 - Choose **Browse S3**
 - Select the radio button next to the **glue-lab-processed-<account-id>-<region>** bucket

- Choose **Choose**
- In the **S3 Target Location** search box append **/orders_enriched/**

to the existing value so the path reads

s3://glue-processed-<account-id>-<region>/orders_enriched/

- **Data Catalog update options:** select **Create a table in the Data Catalog and on subsequent runs, update the schema** and add new partitions.
- **Database:** select **glue_db**
- **Table name:** paste or type in **orders_enriched**
- **Partition Keys - optional:**
 - Choose **Add a partition key**
 - From the drop down menu, choose **sales_region**.

Note: Partitioning by **sales_region** writes the output into one subfolder per sales region. Queries that filter by sales region then scan only the matching partition - a common optimization for larger datasets.

Partitioning is a best practice once a dataset grows large and is queried repeatedly: pick a column you filter on often that has a small number of distinct values (such as region, year, or month) so engines skip the partitions that do not match and scan far less data, which lowers query cost and time. Avoid partitioning on high-cardinality columns like **order_id**, which would create many tiny files and slow queries down. For guidance, see [Managing partitions for ETL output in AWS Glue](#).

Note: Leave **Data Quality** turned off for this job. AWS Glue Data Quality result publishing is not available in this lab environment, and the pipeline does not require it.

36. Choose **Save**.

Task 2.5: Enable job bookmarks

Turn on the job bookmark so AWS Glue can track which source data it has already processed.

37. Near the top of the page, choose the **Job details** tab, then scroll down and set:

- **Job bookmark:** **Enable**

Note: A **job bookmark** lets AWS Glue track which source data it has already processed so future runs handle only new data. You enable it now; you will see the payoff in Task 4.

Task 2.6: Bookmark only the orders (fact) table

A job bookmark tracks *every* source in the job. With it on, the next run will skip not only the orders already processed but also the entire **products** table - Glue treats the unchanged dimension table as “already done” and reads zero rows from it. Because the join is an inner join, an empty products side produces zero output rows, and the pipeline silently stops adding data. The fix is the standard pattern for this case: **bookmark only the incremental fact table (orders), and let the static**

dimension table (products) read in full on every run. Glue Studio has no per-source bookmark switch, so you make this one change in the job's script.

38. At the top of the canvas, choose the **Script** tab, then choose **Edit script**. A dialog warns that you can no longer edit this job visually - choose **Confirm**.

39. Find the line (around line 24) that reads the **products** source (your node id will differ):

```
ProductsDataCatalog_nodeXXXXX =  
glueContext.create_dynamic_frame.from_catalog(database="glue_lab_db", table_name="products",  
transformation_ctx="ProductsDataCatalog_node1")
```

Delete just the

```
, transformation_ctx="ProductsDataCatalog_node####"
```

portion - from the comma through the closing quote - so the line ends at

```
table_name="products")
```

. Leave the **orders** source line and its

```
transformation_ctx
```

exactly as they are.

Analysis: A source is bookmarked only when it carries a

```
transformation_ctx
```

. Removing it from **products** makes that source read fresh on every run, while **orders** keeps its

```
transformation_ctx
```

and stays incremental. Bookmarking only the fact source is the standard way to join incremental data against static reference tables.

40. Choose **Save**, then choose **Run**.

41. Choose the **Runs** tab and wait for **Run status** to change from *Running* to *Succeeded*.

Note: the first run typically takes two to three minutes. You can optionally use the refresh icon to refresh the view and notice more quickly when it succeeded.

Task 2.7: Confirm the Parquet output landed in Amazon S3

Browse the processed bucket to see the partitioned Parquet files the job wrote.

42. In the AWS Management Console search box, enter **S3**

and choose **S3**. Open the **glue-processed-<account-id>** bucket, then open the **orders_enriched/** folder.

Analysis: Instead of a single flat file, the output is split into one **sales_region=** subfolder per sales region. This is the partitioning you configured in Task 2.4 - Glue writes each partition key as its own S3 prefix so query engines can skip irrelevant data.

43. Open any **sales_region=** subfolder and confirm the data files exist and that they end in **.snappy.parquet**.

Analysis: The job wrote columnar, Snappy-compressed **Parquet**, not CSV. The raw **orders/** and **products/** CSV sources are untouched - this enriched copy is a separate, query-optimized dataset.

Task 3: Orchestrate the pipeline with a Glue Workflow

So far you have run the crawler and the job by hand - exactly the manual, error-prone process AnyCompany wants to eliminate. A **Workflow** lets the entire pipeline run as one unit: every step executes in the same order every time, with no analyst having to remember to run the crawler before the job. That consistency is what removes human error and frees the team from repetitive manual work. In this task, you build an AWS Glue **Workflow** that runs the crawler and then the ETL job automatically, using **triggers** to connect them.

Task 3.1: Create the workflow

Create an empty Workflow that will tie the crawler and ETL job together.

52. Return to the AWS Glue console.
53. In the AWS Glue navigation pane, under **Data Integration and ETL**, choose **Workflows (orchestration)**.
54. Choose **Add workflow**.
55. For **Workflow name**, enter **orders-pipeline-workflow**
56. Choose **Create workflow**.

The empty workflow is created and its details page opens.

Task 3.2: Add a start trigger and the crawler

Add an on-demand trigger that starts the workflow by running the crawler.

57. Choose the **orders-pipeline-workflow** workflow link to open its graph view.
58. In the **Graph** tab, choose **Add trigger**. In the dialog, choose the **Add new** tab and configure:
 - **Name:** start-trigger
 - **Trigger type:** On demand
 - Choose **Add**.

The trigger appears on the canvas.

59. Choose the **Add node** box to the right of start-trigger.
60. Choose the **Crawlers** tab, then select **raw-data-crawler**, and choose **Add**.

This wires the on-demand trigger to start the crawler.

Task 3.3: Add an event trigger and the ETL job

Add a trigger that runs the ETL job automatically once the crawler finishes.

61. Select the **raw-data-crawler** node on the canvas.

A new Add trigger item appears on the canvas.

62. Choose **Add trigger**.

63. On the **Add new** tab, configure:

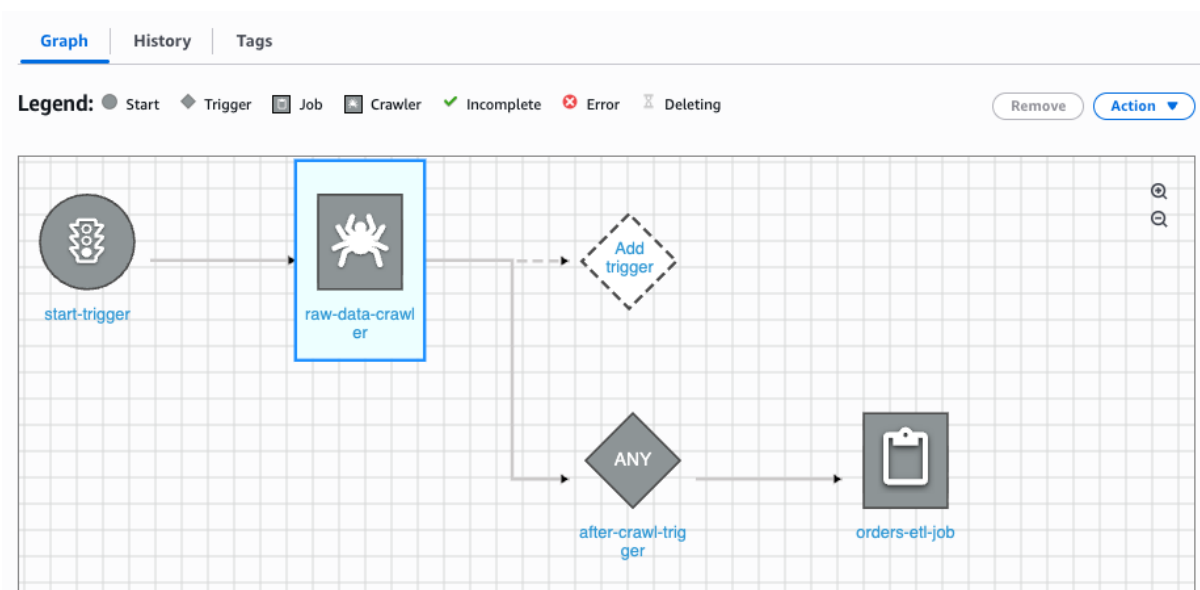
- **Name:** replace the default name with **after-crawl-trigger**
- **Trigger type:** Event
- **Trigger logic:** Start after ANY watched event
- Choose **Add**.

Note: This event trigger fires when the crawler it follows completes successfully, chaining the next step automatically.

64. To the **right** of the new trigger, choose **Add node**.

65. Choose the **Jobs** tab, select **orders-etl-job**, and then choose **Add**.

Your workflow graph now reads: **start-trigger** → **raw-data-crawler** → **after-crawl-trigger** → **orders-etl-job**.



Task 3.4: Run the workflow

Run the whole pipeline with one action and watch the bookmark skip already-processed data.

66. At the top of the workflow page, choose **Run workflow**.

67. Choose the **History** tab, select the row in the Workflow runs panel and choose **View run details**.

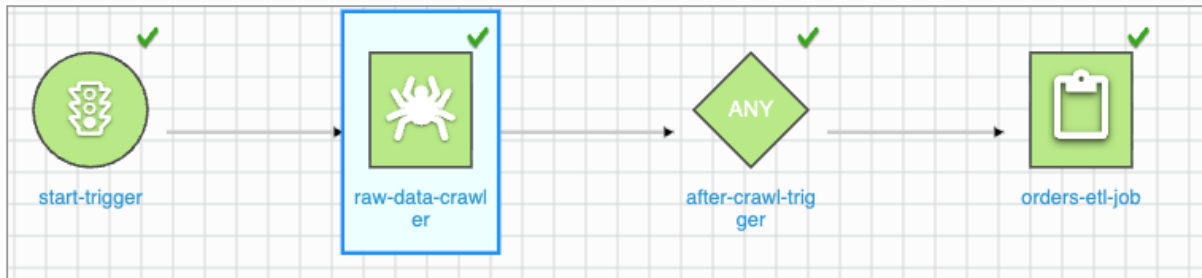
68. Watch the graph progress as each node in the workflow completes. Wait until all 4 items in the graph show the green checkmark. It can take as long as 6 minutes to complete.

Warning: Wait until the workflow completes before moving on to the next step.

Note: Optionally choose the refresh icon in the top right corner of the page on occasion to refresh the view.

Legend:

✓ Succeeded 🔄 Running ✗ Stopped ✗ Failed



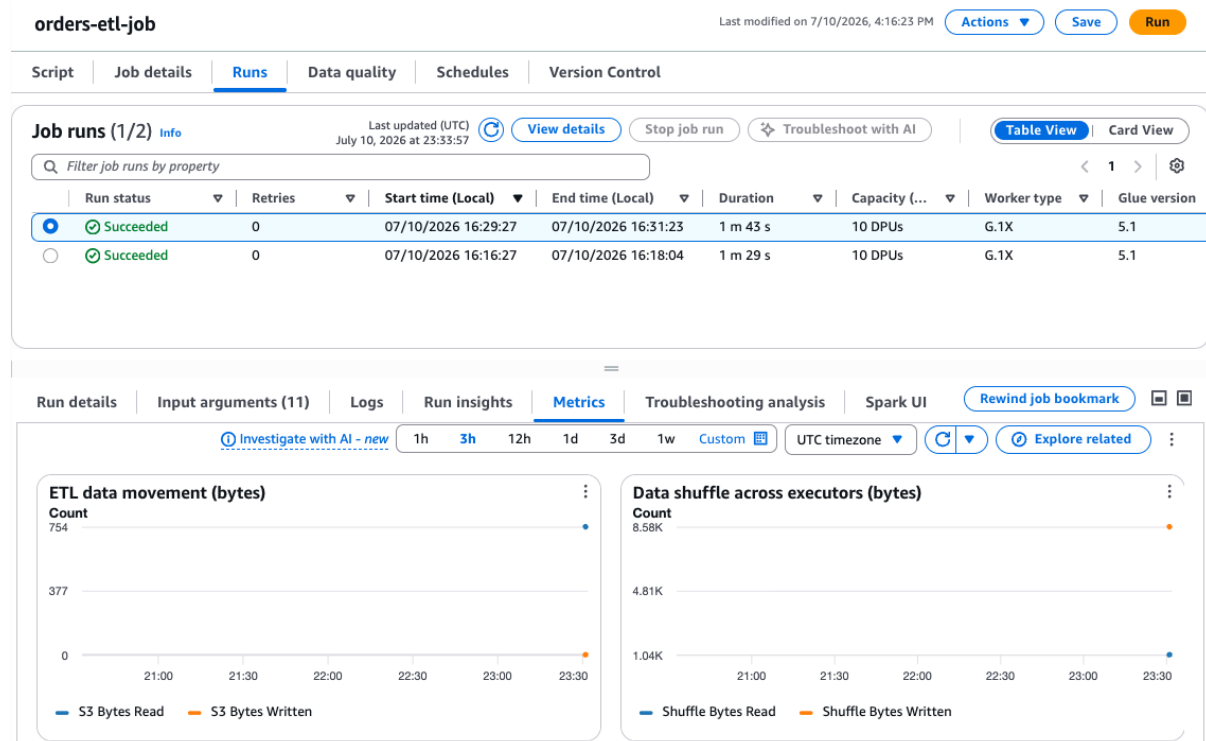
Note: The legend above the graphs shows the meaning of the icons that appear at the top of each node in the workflow as the run progresses. It will take a few minutes to complete.

Analysis: The entire pipeline ran from a single action: the start trigger launched the crawler, and when the crawler finished, the event trigger automatically handed off to the ETL job. No manual steps were required.

69. Return to the Glue console's **Visual ETL** page and select the **orders-etl-job** job.

70. Choose **Runs** and then select the most recent run - this is the run the workflow just launched, and it is selected by default.

71. In the run details area below, choose the **Metrics** tab.



On the **ETL data movement** chart, notice that **S3 Bytes Written** (the orange dot) sits at **0**, while **S3 Bytes Read** (the blue dot) is not zero.

Analysis: Zero bytes written is what you see, because of the job bookmark: the latest run found no new **orders** to process, so it produced no new output. It still *reads* some data because the **products** dimension is not bookmarked and is scanned on every run. Recall that you already manually processed the ETL job once before creating the pipeline and running it just now. You will analyze that earlier run next.

72. Back in the **Job runs** list above, select the *previous* run - the one you ran manually back in Task 2.

73. Notice that during this earlier run, the same **ETL data movement** chart shows that S3 bytes *were* written.

Analysis: The difference comes from the job bookmark you enabled in Task 2. This first run processed the data and wrote its output. But when the more recent job ran, the bookmark remembered that work, so when the workflow ran it found no new data and wrote nothing. That is exactly the behavior you want: re-running the pipeline does not reprocess old data.

Task 4: Process new data incrementally

AnyCompany's orders don't arrive all at once - new ones land every day. A pipeline that reprocessed the entire order history on every run would get slower and more expensive as the business grows. In this task, a new batch of orders arrives, you re-run the same workflow, and the **job bookmarks** setting you applied earlier ensures that only new records are processed. AnyCompany gets fresh, complete insights on every run while paying only to process what actually changed.

Task 4.1: Promote the delta batch into the orders folder

Move a new batch of orders into the folder the crawler scans.

77. In the console search box, enter S3 and choose **S3**. Open the **glue-raw-<account-id>-<region>** bucket.

78. Open the **incoming/** folder, select the checkbox next to **orders-batch2.csv**

79. Choose **Actions**, and then choose **Copy**.

80. For the destination:

- Choose **Browse S3**
- Choose the same **glue-raw...** bucket in the breadcrumb links
- Select the radio button next to the **orders/** folder
- Choose **Choose destination**, and then choose **Copy**.
- Confirm you see a *"Successfully copied objects"* banner, then choose **Close**.

81. Confirm:

- Choose the **glue-raw...** link in the breadcrumb navigation above, then choose the orders link.
- The orders S3 prefix (folder) should now show two objects in it (the **orders.csv** that was there before, as well as the **orders-batch2.csv** file you just copied there).

Note: Because the **orders** table defined in AWS Glue points at the whole **orders/** prefix, no schema change is needed - the table automatically covers the new file.

Task 4.2: Re-run the workflow

Run the same workflow again so the new batch flows through the pipeline.

82. Return to **AWS Glue > Workflows (orchestration)** area in the Glue console.
83. Choose the **orders-pipeline-workflow** link.
84. Choose **Run workflow**.
85. On the **History** tab, select the new run (which has status “Running”), then select **View run details**
86. Wait until all nodes turn green.

Note: As was the case before, it will take a few minutes for the pipeline run to complete. You may want to choose the refresh icon in the top right corner of the page on occasion to refresh the view.

Do not move on to the next step until the run details attains the *Completed* status.

Task 5: Visualize the result in a Glue notebook

To close the loop from raw data to insight, you open an AWS Glue **interactive notebook** - a serverless Jupyter notebook backed by an AWS Glue session - read the processed table with PySpark, and chart it. The result is exactly the revenue-by-supplier view Mateo wanted but could not easily produce from the raw, unjoined files.

Task 5.1: Upload the notebook

Rather than write the analysis code by hand, you will use a ready-made AWS Glue notebook that already contains it. First download the notebook from Amazon S3, then upload it to AWS Glue Studio.

88. Download the notebook file:
 - In the AWS Management Console, open the **Amazon S3** console.
 - Choose the **glue-raw-<account-id>-<region>** bucket, open the **notebook/** folder, select **order-analysis.ipynb**.
 - Choose **Object actions**, and then select **Download as**.
 - Next, right-click on **order-analysis.ipynb** and choose **Save link as**.

Warning: Your browser may try to change the file extension to .json. Be sure that the Save As dialog names the file order-analysis.ipynb. Rename it back to .ipynb if necessary.

- Choose **Save**.
89. In the AWS Management Console search box, enter Glue and choose **AWS Glue**.

90. In the navigation pane, under **Data Integration and ETL**, choose **Notebooks**.

91. Choose **Notebook**.

92. For the options, select **Upload notebook**, choose **Choose file**, select the **order-analysis.ipynb** file you just downloaded, and choose **Open**.

93. For **IAM role**, choose **glue-lab-service-role**, and then choose **Create notebook**.

Note: It takes a minute or two for the notebook to open. When it does, it already contains three code cells - session setup, read the data, and plot the chart. You do not need to add or edit any code; you will simply run each cell in order.

94. Choose the notebook name at the top and rename it to **orders-analysis**.

Task 5.2: Start the session and read the data

Run the notebook cells in order. The first cell starts a small, low-cost AWS Glue interactive session; the second reads your enriched table.

95. Select the first code cell (it begins with `%idle_timeout`) and run it to start the session.

Note: To run a cell, place your cursor in it and choose the play (triangle) icon at the top of the notebook. While a cell runs, `[*]` appears to its left; when it finishes, a number such as `[1]` appears. The numbers increment top to bottom so you can see which cells have run.

Note: The session is deliberately small (

`%idle_timeout 30, %number_of_workers 2`) so it does not keep billing after you finish. Starting it takes one to two minutes; when it is ready the cell prints a session ID. `matplotlib` and `pandas` are already available in the session, so no extra install is needed.

**** EXAMPLE OUTPUT ****

Current `idle_timeout` is 30 minutes.

`idle_timeout` has been set to 30 minutes.

Setting Glue version to: 5.1

Previous worker type: G.1X

Setting new worker type to: G.1X

Previous number of workers: 2

Setting new number of workers to: 2

Trying to create a Glue session for the kernel.

Session Type: glueetl

Worker Type: G.1X

Number of Workers: 2

Idle Timeout: 30

Session ID: d3493089-82b8-4c63-a375-9da330aa0e94

Applying the following default arguments:

```
--glue_kernel_version 1.0.10
```

```
--enable-glue-datacatalog true
```

Waiting for session d3493089-82b8-4c63-a375-9da330aa0e94 to get into ready status...

Session d3493089-82b8-4c63-a375-9da330aa0e94 has been created.

96. Select the second cell and run it. It reads the **orders_enriched** table and aggregates revenue by supplier:

97. import matplotlib.pyplot as plt

98.

99. df = spark.sql("""

100. SELECT supplier, SUM(quantity * unit_price) AS revenue

101. FROM glue_lab_db.orders_enriched

102. GROUP BY supplier

103. ORDER BY revenue DESC

104. """)

105. pdf = df.toPandas()

pdf

It takes about 30 seconds; the aggregated results print as a table below the cell.

Task 5.3: Plot the result

Render the aggregated revenue as a bar chart.

97. Select the third cell and run it to build and display the chart:

98. ax = pdf.plot(kind="bar", x="supplier", y="revenue", legend=False, figsize=(5, 5))

99. ax.set_title("AnyCompany revenue by supplier")

100. ax.set_ylabel("Revenue (USD)")

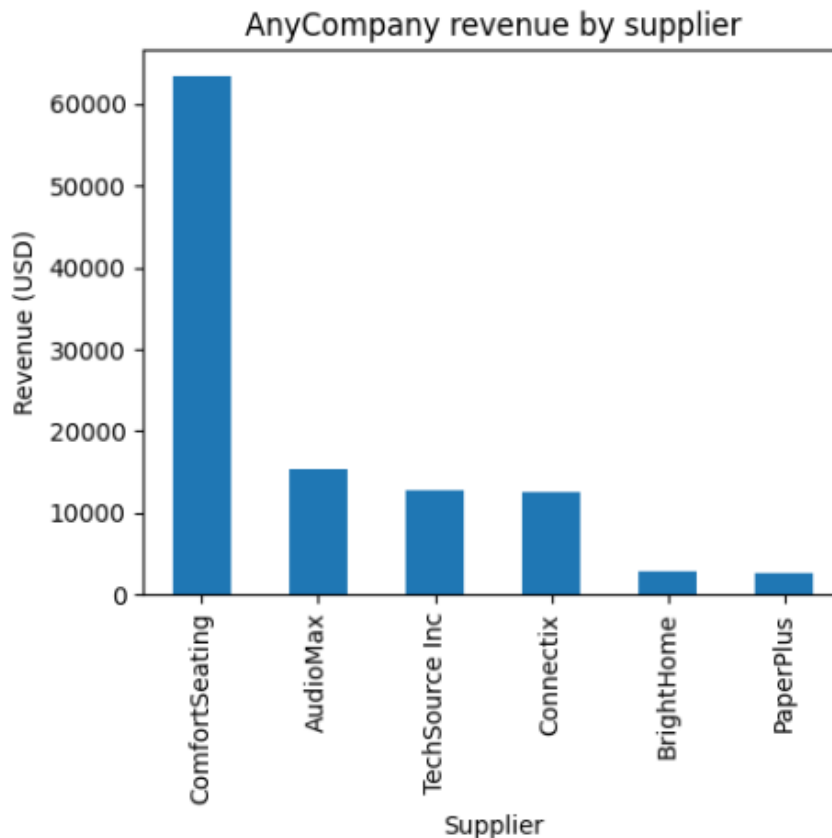
101. ax.set_xlabel("Supplier")

102. plt.tight_layout()

%matplotlib plt

98. The bar chart displays below the cell.

```
[12]: ax = pdf.plot(kind="bar", x="supplier", y="revenue", legend=False, figsize=(5, 5))
ax.set_title("AnyCompany revenue by supplier")
ax.set_ylabel("Revenue (USD)")
ax.set_xlabel("Supplier")
plt.tight_layout()
%matplotlib plt
```



Analysis: The code read from the same processed table your pipeline produced and turned it into a chart - all within a serverless AWS Glue session, no separate analytics service required. This is the consumption layer of the pipeline you built.

Note: This chart is made possible because of the **join** you built in Task 2. Revenue comes from the **orders** data (quantity and price per line), but the **supplier** label lives in the **products** data - neither table could produce this chart on its own. When the ETL job joined **orders** to **products** on **product_name**, it wrote both together into the **orders_enriched** table, so a single query can now group revenue by supplier. The chart is the visual payoff of that earlier join.

Note: Because the pipeline is incremental, keeping this chart current is simple. When new orders land (as you saw in Task 4, when **orders-batch2.csv** was promoted into **orders/**), the workflow re-runs and the **job bookmark** processes only the new rows - it joins just those new orders against **products** and appends them to **orders_enriched**, without reprocessing everything. To refresh the chart, you simply re-run the read cell and the chart cell: the query picks up the larger **orders_enriched** table and the bars update to reflect the new totals. That is the full loop - new data in, delta processed, insight refreshed.

Warning: AWS Glue interactive sessions bill for the time they run. When you finish, stop the session (or let it time out via the `%idle_timeout` you set) so it does not keep running.

Task complete: You visualized the pipeline output in an AWS Glue interactive notebook.

Conclusion

You successfully did the following:

- Cataloged two raw datasets and **joined** them in an AWS Glue Visual ETL job to produce an enriched, partitioned table.
- **Orchestrated** the crawler and ETL job as a single AWS Glue Workflow using triggers.
- Processed a new data batch **incrementally** with AWS Glue job bookmarks.
- **Visualized** the result in an AWS Glue interactive notebook.

AnyCompany has replaced Mateo's slow, error-prone spreadsheet with an automated AWS Glue pipeline. The business now gets consistent, up-to-date insights - such as the revenue-by-supplier chart Mateo used to build by hand - on every run, with new orders processed efficiently as they arrive.